

Computational Physics 1

Youssef Tawfik

Problem 1

Created an empty file named exp.cpp

```
#include <cmath>

int main (int argc, char ** argv) {
    return 0;
}
```

Ran the four utilities which had the following outputs. ldd shows the libc.so.6 library being linked. passing nm exp through c++filt doesn't do anything, I assume that is because there is nothing to demangle.

```
$ ldd exp
linux-vdso.so.1 (0x00007ffc815eb000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x0000793d82e00000)
/lib64/ld-linux-x86-64.so.2 (0x0000793d83061000)
```

```
$ file exp
exp: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, inter-
preter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=f86c8dd4409f23226ca892edc4a0f62a4df170dc, for
GNU/Linux 3.2.0, not stripped
```

```

$nm exp | c++filt
000000000000038c r __abi_tag
0000000000004010 B __bss_start
0000000000004010 b completed.0
w __cxa_finalize@GLIBC_2.2.5
0000000000004000 D __data_start
0000000000004000 W data_start
0000000000001070 t deregister_tm_clones
00000000000010e0 t __do_global_dtors_aux
0000000000003df8 d __do_global_dtors_aux_fini_array_entry
0000000000004008 D __dso_handle
0000000000003e00 d _DYNAMIC
0000000000004010 D _edata
0000000000004018 B _end
0000000000001140 T _fini
0000000000001120 t frame_dummy
0000000000003df0 d __frame_dummy_init_array_entry
00000000000020c0 r __FRAME_END__
0000000000003fc0 d _GLOBAL_OFFSET_TABLE_
w __gmon_start__
0000000000002004 r __GNU_EH_FRAME_HDR
0000000000001000 T _init
0000000000002000 R _IO_stdin_used
w _ITM_deregisterTMCloneTable
w _ITM_registerTMCloneTable
U __libc_start_main@GLIBC_2.34
0000000000001129 T main
00000000000010a0 t register_tm_clones
0000000000001040 T _start
0000000000004010 D __TMC_END__

```

After including the math library to the file header the outputs of ldd and nm seem unchanged. If however I run exp with a variable argument I get a different result for ldd

```

ldd exp
linux-vdso.so.1 (0x00007ffc6a307000)
libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6 (0x000078496b830000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x000078496b600000)
/lib64/ld-linux-x86-64.so.2 (0x000078496b938000)

```

For the first output we only had a single library linked (libc.so.6) but when the math library is called on a variable input the library (libm.so.6) is also linked.

Problem 2

Using the sstream library I convert the command line argument to a string and feed it into the exp function and generate the required output.

```
$ ./exp 5
148.413
```

The output of `ldd` shows two extra libraries being linked. The output of `nm` grows substantially and `c++filt` demangles its output.

Problem 3

I defined the routine `exp_taylor()` in the main function which takes a value of x and the number of expansion terms to produce the following output.

```
$ ./exp
-10 | 4.53999e-05
-8 | 0.000335463
-6 | 0.00247875
-4 | 0.0183156
-2 | 0.135335
0 | 1
2 | 7.38906
4 | 54.5982
6 | 403.429
8 | 2980.96
10 | 22026.5
```

We can use `nm` to determine whether the routine `exp()` is being called from inside the executable program file by looking for the function in the `nm` list.

```
Using exp():
                U exp@GLIBC_2.2.9
Using exp_taylor():
0000000000001214 T _Z10exp_taylordi
```

The first column is the address which we can see is not shown for `exp()` since it is linked from an external library. The second column is the type which for the local routine is `T` and for the external routine is `U` (undefined).

Problem 4

In the command line `/path/to/foo*` expands into all files in the directory `/path/to/` whose name begins with `foo`.

Problem 5

An instance of running the program.

```
$ ./plus 1 5
6
```

Problem 6

Created a library libMain.a built by the object code for the parse add and print routines. This program generates the same output as that of Problem 5.

Problem 7

To create a shared library I used the g++ compiler with a -shared flag. Under the default target plus, I added the flag -Wl,-rpath,. to the compiler command. This is because during the program execution the compiler doesn't automatically check the working directory for the shared library. This was discussed under the "Mysterious Problems" section, but I didn't know if there was another standard way of fixing this problem.

Running ldd with and without the flag -Wl,-rpath,. the issue becomes clear

```
Without Flag:  
libMain.so => not found  
With Flag:  
libMain.so => ./libMain.so (0x000071002edf2000)
```

The file size of the executable main is 17.2 kB and 20.4 kB for the shared and static library respectively.

Problem 8

Make files were added to all directories.

Problem 9

All changes pushed to the repository <https://github.com/ytawfik23/ComputationalMethods.git>